

DOCUMENTACION TECNICA

Para Desarrolladores y Administradores del
Sistema

Framework: Laravel 9.52

Version: 1.0

Fecha: 25/11/2024

Sistema: E-Commerce B2B + Servicio Tecnico

1. RESUMEN EJECUTIVO

1.1 Propósito del Sistema

Este sistema Laravel 9 integra dos líneas de negocio principales:

- E-Commerce B2B:** Catálogo de productos con sistema de cotizaciones, gestión de inventario y acceso temporal por enlaces tokenizados
- Gestión de Servicio Técnico:** Administración completa de órdenes de servicio para equipos de seguridad (CCTV, control de acceso)

1.2 Usuarios Objetivo

- Administradores:** Acceso total al sistema
- Vendedores:** Gestión de catálogo, clientes y cotizaciones B2B
- Técnicos:** Gestión de órdenes de servicio y reparaciones

1.3 Características Principales

- Catálogo de productos con variantes (talla/color)
- Sistema de precios multi-nivel (6 listas de precios)
- Control de inventario con trazabilidad completa
- Generación de enlaces temporales para acceso de clientes
- Gestión de órdenes de servicio técnico
- Registro de equipos y repuestos
- Generación de PDFs y exportación a Excel
- Dashboard con métricas y KPIs

2. ARQUITECTURA DEL SISTEMA

2.1 Stack Tecnológico

Backend

- Framework:** Laravel 9.52
- PHP:** 8.0.2+
- Base de Datos:** MySQL (via XAMPP)
- Servidor Web:** Apache (XAMPP)

Frontend

- Motor de Plantillas:** Blade
- CSS:** Tailwind CSS 3.4.17 + Bootstrap 5.3.7 (híbrido)

- **JavaScript:**
 - Alpine.js 3.4.2 (componentes reactivos)
 - jQuery 3.7.1 (requerido por DataTables)
 - Livewire 2.12 (componentes dinámicos)

Librerías Clave

- **Autenticación:** Laravel Breeze
- **Permisos:** Spatie Laravel Permission
- **DataTables:** Yajra DataTables 10.0 (paginación server-side)
- **Excel:** Maatwebsite Excel 3.1 + PHPSpreadsheet 1.29.7
- **PDF:** Barryvdh DomPDF 3.1
- **Notificaciones:** SweetAlert2 11.22

Build Tools

- **Bundler:** Vite 4.0
- **CSS Processors:** PostCSS, Autoprefixer

2.2 Patrón de Arquitectura

MVC (Model-View-Controller) con las siguientes capas:

```
app/
|--?? Http/
|   |--?? Controllers/           # Controladores
|   |   |--?? ProductosController.php
|   |   |--?? StockController.php
|   |   |--?? CatalogoController.php
|   |   |--?? SolicitudController.php
|   |   |--?? ServicioTecnico/   # Namespace para módulo ST
|--?? Models/                   # Modelos Eloquent (30+)
|--?? Imports/                  # Clases de importación Excel
|--?? Exports/                  # Clases de exportación Excel
|--?? Mail/                     # Plantillas de email

resources/
|--?? views/                    # Vistas Blade
|   |--?? productos/
|   |--?? catalogo/
|   |--?? stock/
|   |--?? solicitudes/
|   |--?? servicio-tecnico/
|   |--?? pdf/
|   |--?? layouts/

routes/
|--?? web.php                   # Definición de rutas

database/
|--?? migrations/               # 40+ archivos de migración
```

2.3 Entorno de Desarrollo

Servicios Requeridos:

1. XAMPP (MySQL + Apache)
2. Laravel Dev Server: `php artisan serve` (puerto 8000)
3. Vite Dev Server: `npm run dev` (puerto 5173, hot reload)

Comandos de Build:

```
# Desarrollo
npm run dev          # Inicia Vite con hot reload
php artisan serve    # Inicia servidor Laravel

# Producción
npm run build        # Compila assets para producción

# Base de datos
php artisan migrate  # Ejecuta migraciones
php artisan migrate:fresh --seed # Base de datos limpia con datos

# Testing
php artisan test
vendor/bin/phpunit
```

3. ESQUEMA DE BASE DE DATOS

3.1 Resumen

- **Total de Tablas:** 40+
- **Relaciones:** Predominantemente 1:N con algunas N:M
- **Convenciones:**
 - Foreign keys: `{tabla_singular}_id`
 - Timestamps: `created_at`, `updated_at`
 - Soft delete: Flag `activo` (no usa Laravel soft deletes)

3.2 Tablas Principales - E-Commerce B2B

3.2.1 Productos y Catálogo

`productos`

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
referencia        VARCHAR(255) UNIQUE NOT NULL
nombre            VARCHAR(255) NOT NULL
descripcion       TEXT NULL
marca             VARCHAR(255) NULL
categoria_id      BIGINT -> categorias.id
unidad_venta      VARCHAR(255) NOT NULL
unidad_empaque    VARCHAR(255) NOT NULL
extension         VARCHAR(255) NULL
activo            BOOLEAN DEFAULT TRUE
tiene_variantes   BOOLEAN DEFAULT FALSE
controlar_stock   BOOLEAN DEFAULT TRUE
permitir_venta_sin_stock BOOLEAN DEFAULT FALSE
eliminado         BOOLEAN DEFAULT FALSE
created_at, updated_at  TIMESTAMP

```

categorias

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
nombre            VARCHAR(255) NOT NULL
slug              VARCHAR(255) UNIQUE NOT NULL
descripcion       TEXT NULL
orden             INTEGER NOT NULL
activo            BOOLEAN DEFAULT TRUE
created_at, updated_at  TIMESTAMP

```

variantes_productos

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
producto_id       BIGINT -> productos.id
talla             VARCHAR(255) NULL
color             VARCHAR(255) NULL
sku               VARCHAR(255) UNIQUE NOT NULL
activo            BOOLEAN DEFAULT TRUE
created_at, updated_at  TIMESTAMP

```

imagenes_producto

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
producto_id       BIGINT -> productos.id
ruta_imagen       VARCHAR(255) NOT NULL
texto_alternativo VARCHAR(255) NULL
es_principal      BOOLEAN DEFAULT FALSE
orden             INTEGER DEFAULT 0
created_at, updated_at  TIMESTAMP

```

3.2.2 Sistema de Precios

listas_precios

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
codigo            VARCHAR(50) UNIQUE NOT NULL -- export1, export2, local1-4
nombre            VARCHAR(255) NOT NULL
descripcion       TEXT NULL
activo            BOOLEAN DEFAULT TRUE
created_at, updated_at  TIMESTAMP

```

precios_productos

```

id          BIGINT PRIMARY KEY AUTO_INCREMENT
producto_id BIGINT -> productos.id
lista_precio_id BIGINT -> listas_precios.id
precio      DECIMAL(10,2) NOT NULL
activo      BOOLEAN DEFAULT TRUE
created_at, updated_at TIMESTAMP
UNIQUE(producto_id, lista_precio_id)

```

precios_variantes

```

id          BIGINT PRIMARY KEY AUTO_INCREMENT
variante_producto_id BIGINT -> variantes_productos.id
lista_precio_id BIGINT -> listas_precios.id
precio      DECIMAL(10,2) NOT NULL
activo      BOOLEAN DEFAULT TRUE
created_at, updated_at TIMESTAMP
UNIQUE(variante_producto_id, lista_precio_id)

```

actualizaciones_precios (auditoría)

```

id          BIGINT PRIMARY KEY AUTO_INCREMENT
usuario_id  BIGINT -> users.id
estado      ENUM('procesando','completado','error')
nombre_archivo VARCHAR(255) NOT NULL
ruta_archivo VARCHAR(255) NOT NULL
total_filas INTEGER NOT NULL
actualizaciones_exitosas INTEGER DEFAULT 0
actualizaciones_fallidas INTEGER DEFAULT 0
errores     JSON NULL
detalles_procesados JSON NULL
created_at, updated_at TIMESTAMP

```

3.2.3 Control de Stock

stock_productos

```

id          BIGINT PRIMARY KEY AUTO_INCREMENT
producto_id BIGINT -> productos.id
variante_producto_id BIGINT -> variantes_productos.id (nullable)
cantidad_disponible INTEGER DEFAULT 0
cantidad_reservada  INTEGER DEFAULT 0
stock_minimo        INTEGER DEFAULT 0
stock_maximo        INTEGER NULL
ubicacion            VARCHAR(255) NULL
alerta_stock_bajo    BOOLEAN DEFAULT TRUE
notas                TEXT NULL
created_at, updated_at TIMESTAMP
UNIQUE(producto_id, variante_producto_id)

```

movimientos_stock

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
producto_id       BIGINT -> productos.id
variante_producto_id BIGINT -> variantes_productos.id (nullable)
tipo_movimiento   ENUM('entrada','salida','ajuste','reserva','liberacion')
cantidad          INTEGER NOT NULL
stock_anterior    INTEGER NOT NULL
stock_nuevo       INTEGER NOT NULL
referencia_documento VARCHAR(255) NULL
origen            ENUM('compra','venta','devolucion','ajuste_inventario','cotizacion')
motivo            TEXT NULL
usuario_id        BIGINT -> users.id
solicitud_cotizacion_id BIGINT -> solicitudes_cotizacion.id (nullable)
created_at, updated_at  TIMESTAMP

```

3.2.4 Clientes y Geografía

clientes (clientes B2B)

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
numero_identificacion VARCHAR(255) UNIQUE NOT NULL
nombre_contacto   VARCHAR(255) NOT NULL
email             VARCHAR(255) UNIQUE NOT NULL
telefono          VARCHAR(100) NULL
pais_id           BIGINT -> paises.id
ciudad_id         BIGINT -> ciudades.id
vendedor_id       BIGINT -> users.id
lista_precio_id   BIGINT -> listas_precios.id
activo            BOOLEAN DEFAULT TRUE
created_at, updated_at  TIMESTAMP

```

paises

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
nombre            VARCHAR(255) NOT NULL
codigo            VARCHAR(10) UNIQUE NOT NULL
created_at, updated_at  TIMESTAMP

```

departamentos

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
pais_id           BIGINT -> paises.id
nombre            VARCHAR(255) NOT NULL
created_at, updated_at  TIMESTAMP

```

ciudades

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
departamento_id BIGINT -> departamentos.id
nombre            VARCHAR(255) NOT NULL
created_at, updated_at  TIMESTAMP

```

3.2.5 Cotizaciones y Enlaces

enlaces_acceso

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
cliente_id        BIGINT -> clientes.id
creado_por        BIGINT -> users.id
token             VARCHAR(255) UNIQUE NOT NULL
dias_validos      INTEGER NOT NULL
mostrar_precios   BOOLEAN DEFAULT TRUE
mostrar_stock     BOOLEAN DEFAULT TRUE
expira_en         TIMESTAMP NOT NULL
activo            BOOLEAN DEFAULT TRUE
visitas           INTEGER DEFAULT 0
ultimo_acceso     TIMESTAMP NULL
notas             TEXT NULL
created_at, updated_at  TIMESTAMP

```

solicitudes_cotizacion

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
numero_solicitud  VARCHAR(255) UNIQUE NOT NULL -- SC-YYYYMMDDHHMMSS-XXXX
cliente_id        BIGINT -> clientes.id
enlace_acceso_id  BIGINT -> enlaces_acceso.id (nullable)
created_by        BIGINT -> users.id (nullable)
estado            ENUM('pendiente','aplicada','rechazada') DEFAULT 'pendiente'
monto_total       DECIMAL(10,2) NOT NULL
notas_cliente     TEXT NULL
observaciones_admin TEXT NULL
motivo_rechazo    TEXT NULL
aplicada_en       TIMESTAMP NULL
aplicada_por      BIGINT -> users.id (nullable)
rechazada_en      TIMESTAMP NULL
rechazada_por     BIGINT -> users.id (nullable)
created_at, updated_at  TIMESTAMP

```

items_solicitud_cotizacion

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
solicitud_cotizacion_id  BIGINT -> solicitudes_cotizacion.id
producto_id       BIGINT -> productos.id
variante_producto_id  BIGINT -> variantes_productos.id (nullable)
cantidad          INTEGER NOT NULL
precio_unitario   DECIMAL(10,2) NOT NULL
precio_total      DECIMAL(10,2) NOT NULL
precio_editado_manualmente  BOOLEAN DEFAULT FALSE
precio_original   DECIMAL(10,2) NULL
referencia_producto  VARCHAR(255) NOT NULL
nombre_producto   VARCHAR(255) NOT NULL
marca_producto    VARCHAR(255) NULL
info_variante      VARCHAR(255) NULL
created_at, updated_at  TIMESTAMP

```

3.3 Tablas Principales - Servicio Técnico

3.3.1 Clientes y Equipos ST

st_clientes


```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
tipo_cliente     ENUM('particular','empresa') NOT NULL
tipo_documento   VARCHAR(20) NOT NULL
numero_documento VARCHAR(50) UNIQUE NOT NULL
nombre_completo  VARCHAR(255) NOT NULL
razon_social     VARCHAR(255) NULL
email            VARCHAR(255) NULL
telefono        VARCHAR(20) NULL
celular          VARCHAR(20) NOT NULL
direccion        TEXT NULL
ciudad           VARCHAR(255) NULL
departamento    VARCHAR(255) NULL
observaciones    TEXT NULL
activo           BOOLEAN DEFAULT TRUE
created_at, updated_at  TIMESTAMP

```

st_equipos

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
st_cliente_id    BIGINT -> st_clientes.id
tipo_equipo      VARCHAR(100) NOT NULL
marca            VARCHAR(100) NULL
modelo           VARCHAR(100) NULL
numero_serie     VARCHAR(100) UNIQUE NOT NULL
mac_address      VARCHAR(50) NULL
ip_address       VARCHAR(50) NULL
especificaciones TEXT NULL
fecha_compra     DATE NULL
fecha_instalacion DATE NULL
en_garantia      BOOLEAN DEFAULT FALSE
vencimiento_garantia DATE NULL
ubicacion_instalacion VARCHAR(255) NULL
estado           ENUM('operativo','en_reparacion','fuera_servicio','en_bodega')
activo           BOOLEAN DEFAULT TRUE
created_at, updated_at  TIMESTAMP

```

3.3.2 Técnicos y Órdenes

st_tecnicos

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
user_id           BIGINT -> users.id (nullable)
codigo            VARCHAR(20) UNIQUE NOT NULL
nombre_completo  VARCHAR(255) NOT NULL
documento         VARCHAR(50) UNIQUE NOT NULL
email            VARCHAR(255) UNIQUE NOT NULL
telefono         VARCHAR(20) NOT NULL
celular          VARCHAR(20) NOT NULL
especialidad     VARCHAR(255) NULL
fecha_ingreso    DATE NULL
certificaciones  TEXT NULL
activo           BOOLEAN DEFAULT TRUE
created_at, updated_at  TIMESTAMP

```

st_ordenes_servicio

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
numero_orden      VARCHAR(255) UNIQUE NOT NULL -- ST-YYYY-XXXXXX
st_cliente_id     BIGINT -> st_clientes.id
st_equipo_id      BIGINT -> st_equipos.id (nullable)
st_tecnico_id     BIGINT -> st_tecnicos.id (nullable)
tipo_servicio     VARCHAR(255) NOT NULL
prioridad         ENUM('baja','media','alta','urgente') DEFAULT 'media'
estado            ENUM('recibida','asignada','en_proceso','pendiente_repuestos',
                        'completada','entregada','cancelada') DEFAULT
'recibida'
descripcion_problema TEXT NOT NULL
accesorios_entregados TEXT NULL
fecha_recepcion   DATE NOT NULL
fecha_asignacion  TIMESTAMP NULL
fecha_inicio_trabajo TIMESTAMP NULL
fecha_finalizacion TIMESTAMP NULL
fecha_promesa_entrega DATE NULL
fecha_entrega     TIMESTAMP NULL
costo_mano_obra   DECIMAL(10,2) DEFAULT 0
costo_repuestos   DECIMAL(10,2) DEFAULT 0
costo_total       DECIMAL(10,2) DEFAULT 0
cliente_notificado BOOLEAN DEFAULT FALSE
observaciones     TEXT NULL
user_id           BIGINT -> users.id -- usuario creador
created_at, updated_at TIMESTAMP

```

st_diagnostics

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
st_orden_servicio_id BIGINT -> st_ordenes_servicio.id
st_tecnico_id     BIGINT -> st_tecnicos.id
fecha_diagnostico TIMESTAMP NOT NULL
fallas_encontradas TEXT NOT NULL
diagnostico_tecnico TEXT NOT NULL
reparaciones_realizadas TEXT NULL
recomendaciones   TEXT NULL
requiere_repuestos BOOLEAN DEFAULT FALSE
repuestos_necesarios TEXT NULL
tiempo_estimado_horas DECIMAL(5,2) NULL
costo_estimado     DECIMAL(10,2) NULL
created_at, updated_at TIMESTAMP

```

3.3.3 Repuestos e Historial

st_repuestos

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
codigo            VARCHAR(50) UNIQUE NOT NULL
nombre            VARCHAR(255) NOT NULL
descripcion       TEXT NULL
categoria         VARCHAR(100) NOT NULL
marca             VARCHAR(100) NULL
modelo_compatible VARCHAR(100) NULL
stock_actual      INTEGER DEFAULT 0
stock_minimo      INTEGER DEFAULT 0
precio_costo      DECIMAL(10,2) NULL
precio_venta      DECIMAL(10,2) NULL
proveedor         VARCHAR(255) NULL
activo            BOOLEAN DEFAULT TRUE
created_at, updated_at TIMESTAMP

```

st_repuestos_usados

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
st_orden_servicio_id  BIGINT -> st_ordenes_servicio.id
st_repuesto_id      BIGINT -> st_repuestos.id
cantidad           INTEGER NOT NULL
precio_unitario     DECIMAL(10,2) NOT NULL
created_at, updated_at  TIMESTAMP

```

st_historial_estados

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
st_orden_servicio_id  BIGINT -> st_ordenes_servicio.id
estado_anterior      VARCHAR(50) NOT NULL
estado_nuevo         VARCHAR(50) NOT NULL
observaciones        TEXT NULL
usuario_id           BIGINT -> users.id
created_at, updated_at  TIMESTAMP

```

st_imagenes_orden

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
st_orden_servicio_id  BIGINT -> st_ordenes_servicio.id
nombre_archivo       VARCHAR(255) NOT NULL
ruta_archivo         VARCHAR(255) NOT NULL
tipo_imagen          ENUM('recepcion','diagnostico','reparacion','entrega')
orden               INTEGER DEFAULT 0
created_at, updated_at  TIMESTAMP

```

3.4 Tablas de Sistema

users

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
name              VARCHAR(255) NOT NULL
email             VARCHAR(255) UNIQUE NOT NULL
email_verified_at  TIMESTAMP NULL
password          VARCHAR(255) NOT NULL
remember_token     VARCHAR(100) NULL
created_at, updated_at  TIMESTAMP

```

roles (Spatie)

```

id                BIGINT PRIMARY KEY AUTO_INCREMENT
name              VARCHAR(255) NOT NULL -- admin, vendedor, tecnico
guard_name        VARCHAR(255) NOT NULL
created_at, updated_at  TIMESTAMP

```

model_has_roles (Spatie)

```

role_id           BIGINT -> roles.id
model_type         VARCHAR(255) NOT NULL -- App\Models\User
model_id           BIGINT NOT NULL
PRIMARY KEY(role_id, model_id, model_type)

```

logs

```

id          BIGINT PRIMARY KEY AUTO_INCREMENT
user_id     BIGINT -> users.id (nullable)
accion      VARCHAR(255) NOT NULL
modulo      VARCHAR(255) NOT NULL
descripcion TEXT NULL
ip_address  VARCHAR(45) NULL
created_at, updated_at TIMESTAMP

```

3.5 Diagramas de Relaciones

Diagrama E-Commerce B2B

```

categorias (1) ???+??-> (N) productos
|
productos (1) ???+??-> (N) variantes_productos
|--??-> (N) imagenes_producto
|--??-> (N) precios_productos
|__??-> (N) stock_productos

listas_precios (1) ?+??-> (N) precios_productos
|--??-> (N) precios_variantes
|__??-> (N) clientes

países (1) ??-> (N) departamentos (1) ??-> (N) ciudades
|
clientes (1) ?????????????????????????????????+??-> (N) enlaces_acceso
|__??-> (N) solicitudes_cotizacion

solicitudes_cotizacion (1) ??-> (N) items_solicitud_cotizacion

```

Diagrama Servicio Técnico

```

st_clientes (1) ??-> (N) st_equipos

st_ordenes_servicio (1) ?+??-> (N) st_diagnosticos
|--??-> (N) st_repuestos_usados
|--??-> (N) st_historial_estados
|__??-> (N) st_imagenes_orden

st_tecnicos (1) ??-> (N) st_ordenes_servicio
st_equipos (1) ??-> (N) st_ordenes_servicio
st_repuestos (1) ??-> (N) st_repuestos_usados

```

4. MÓDULOS DEL SISTEMA

4.1 Inicio (Dashboard)

Ruta: `/dashboard` **Controlador:** `HomeController` **Acceso:** Todos los usuarios autenticados

Funcionalidad:

- Panel principal personalizado por rol
- Acceso rápido a módulos disponibles
- Notificaciones y alertas

- Últimas actividades

4.2 Servicio Técnico

4.2.1 Dashboard ST

Ruta: `/servicio-tecnico/dashboard` **Controlador:** `ServicioTecnico\DashboardSTController`

Acceso: Admin, Técnico

KPIs Mostrados:

- Total de órdenes activas
- Órdenes por estado (recibida, asignada, en_proceso, completada)
- Órdenes urgentes pendientes
- Técnicos disponibles
- Equipos en reparación
- Stock bajo de repuestos
- Gráficos de tendencias mensuales

4.2.2 Órdenes de Servicio

Ruta: `/servicio-tecnico/ordenes` **Controlador:** `ServicioTecnico\STOrdenServicioController`

Acceso: Admin, Técnico

Funcionalidades:

- Crear nueva orden de servicio
- Ver listado con filtros (estado, prioridad, técnico, fecha)
- Ver detalle de orden
- Cambiar estado (workflow)
- Asignar/reasignar técnico
- Agregar diagnósticos
- Agregar repuestos usados
- Subir imágenes (recepción, diagnóstico, reparación, entrega)
- Generar PDF de orden
- Notificar cliente por email

Estados del Workflow:

1. recibida -> 2. asignada -> 3. en_proceso -> 4. pendiente_repuestos (opcional) ->
2. completada -> 6. entregada | cancelada

4.2.3 Técnicos

Ruta: `/servicio-tecnico/tecnicos` **Controlador:** `ServicioTecnico\STTecnicoController`

Acceso: Admin, Técnico

Funcionalidades:

- CRUD de técnicos
- Auto-creación de cuenta de usuario (rol: tecnico)

- Asignación de especialidades
- Vista de órdenes asignadas
- Historial de trabajo

4.2.4 Clientes ST

Ruta: `/servicio-tecnico/clientes` **Controlador:** `ServicioTecnico\STClienteController`
Acceso: Admin, Técnico

Funcionalidades:

- CRUD de clientes de servicio técnico
- Diferenciación entre particular/empresa
- Vista de equipos del cliente
- Historial de órdenes del cliente

4.2.5 Equipos

Ruta: `/servicio-tecnico/equipos` **Controlador:** `ServicioTecnico\STEquipoController` **Acceso:** Admin, Técnico

Funcionalidades:

- CRUD de equipos de seguridad
- Registro de número de serie único
- Control de garantía
- Estados operacionales
- Historial de servicio por equipo
- Especificaciones técnicas

4.2.6 Repuestos

Ruta: `/servicio-tecnico/repuestos` **Controlador:** `ServicioTecnico\STRepuestoController`
Acceso: Admin, Técnico

Funcionalidades:

- CRUD de repuestos
- Control de inventario
- Alertas de stock bajo
- Historial de uso
- Precios de compra/venta
- Proveedores

4.3 Métricas

Ruta: `/dashboard-metricas` **Controlador:** `DashboardMetricasController` **Acceso:** Admin únicamente

Métricas Mostradas:

- Valor total cotizado

- Cotizaciones aplicadas vs rechazadas vs pendientes
- Tasa de conversión
- Valor cotizado por vendedor
- Gráficos de tendencias
- Filtros por fecha y vendedor

4.4 Usuarios

Ruta: `/usuarios` **Controlador:** `UsuariosController` **Acceso:** Admin únicamente

Funcionalidades:

- CRUD de usuarios
- Asignación de roles (admin, vendedor, tecnico)
- Gestión de contraseñas
- Activar/desactivar usuarios
- Importación masiva de usuarios

4.5 Categorías

Ruta: `/categorias` **Controlador:** `CategoriasController` **Acceso:** Admin únicamente

Funcionalidades:

- CRUD de categorías de productos
- Orden de visualización
- Slugs para SEO
- Activar/desactivar categorías

4.6 Productos

Ruta: `/productos` **Controlador:** `ProductosController` **Acceso:** Admin únicamente

Funcionalidades:

- CRUD de productos
- Gestión de variantes (talla/color)
- Múltiples imágenes por producto
- Precios por lista (6 listas)
- Configuración de control de stock
- Importación masiva de productos (Excel/CSV)
- Actualización masiva de precios (Excel/CSV)
- Modales AJAX para:
 - Ver variantes
 - Ver imágenes
 - Ver precios
 - Ver stock actual

- Historial de actualizaciones de precios
- Soft delete (flag `eliminado`)

4.7 Cotizaciones

Ruta: `/solicitudes` **Controlador:** `SolicitudController` **Acceso:** Admin, Vendedor

Funcionalidades:

- Listado de cotizaciones con filtros
- Ver detalle de cotización
- Aprobar cotización (marca como "aplicada")
 - Opción de descontar stock
 - Registra usuario y fecha
- Rechazar cotización (marca como "rechazada")
 - Requiere motivo
 - Registra usuario y fecha
- Generar PDF (formato Excel)
- Exportar listado a Excel
- Enviar notificación por email al cliente

Estados:

- pendiente: Nueva cotización sin revisar
- aplicada: Cotización aprobada/aceptada
- rechazada: Cotización rechazada

4.8 Clientes

Ruta: `/clientes` **Controlador:** `ClientesController` **Acceso:** Admin, Vendedor

Funcionalidades:

- CRUD de clientes B2B
- Asignación a vendedor
- Asignación de lista de precios
- Ubicación geográfica (país, departamento, ciudad)
- Vista de enlaces generados
- Vista de cotizaciones del cliente
- Filtro por vendedor (vendedores solo ven sus clientes)

4.9 Catálogo

Ruta: `/catalogo` **Controlador:** `CatalogoController` **Acceso:** Admin, Vendedor (autenticado) | Clientes (vía token)

Dos Flujos de Acceso:

Flujo A: Acceso Público (Token)

Ruta: `/catalogo/{token}` **Usuarios:** Clientes sin autenticación

Proceso:

1. Cliente recibe email con enlace tokenizado
2. Accede al catálogo con visibilidad configurada (precios/stock)
3. Navega categorías y productos
4. Agrega productos al carrito
5. Envía solicitud de cotización
6. Sistema registra visita y genera número de solicitud

Flujo B: Acceso Autenticado (Tienda a Tienda)

Ruta: `/catalogo` -> selección de cliente -> `/catalogo/cliente` **Usuarios:** Vendedores y administradores

Proceso:

1. Vendedor selecciona cliente de su cartera
2. Sistema carga catálogo con precios del cliente
3. Vendedor crea cotización en nombre del cliente
4. Puede editar precios manualmente
5. Envía cotización (queda registrado vendedor creador)

4.10 Links (Enlaces de Acceso)

Ruta: `/enlaces` **Controlador:** `EnlacesController` **Acceso:** Admin, Vendedor

Funcionalidades:

- Crear enlace temporal para cliente
- Configuración:
 - Días de validez (1-365)
 - Mostrar/ocultar precios
 - Mostrar/ocultar stock
 - Notas internas
- Ver listado de enlaces (activos/expirados)
- Ver detalle de enlace:
 - URL completa
 - Fecha de expiración
 - Número de visitas
 - Último acceso
 - Cotizaciones generadas
- Activar/desactivar enlace
- Regenerar token

- Enviar por email al cliente

Token: Cadena aleatoria de 32 caracteres, único en la base de datos

4.11 Gestión de Stock

Ruta: `/stock` **Controlador:** `StockController` **Acceso:** Admin, Vendedor

Funcionalidades:

Dashboard de Stock

- Productos con stock bajo
- Productos sin stock
- Movimientos del mes (entradas/salidas)
- Top productos con más rotación
- Productos críticos

Operaciones

- **Entrada de Stock:**
 - Tipo: compra, devolucion
 - Campos: producto/variante, cantidad, ubicación, referencia documento, motivo
 - Registra movimiento con stock_anterior -> stock_nuevo
- **Salida de Stock:**
 - Tipo: venta, transferencia
 - Campos: producto/variante, cantidad, referencia documento, motivo
 - Valida stock disponible
- **Ajuste de Stock:**
 - Tipo: ajuste_inventario
 - Corrige discrepancias
 - Puede aumentar o disminuir

Historial

- Listado completo de movimientos
- Filtros: producto, variante, tipo, fecha, usuario
- Exportar a Excel
- Búsqueda de productos por referencia/nombre (AJAX)

Configuración

- Configurar parámetros de stock por producto:
 - stock_minimo

- stock_maximo
- ubicacion
- alerta_stock_bajo (activar/desactivar)

5. API Y ENDPOINTS

5.1 Endpoints AJAX - Productos

```
GET /productos/{producto}/variantes-ajax
Retorna: HTML con tabla de variantes del producto
Uso: Modal en index de productos

GET /productos/{producto}/imagenes-ajax
Retorna: HTML con galeria de imágenes del producto
Uso: Modal en index de productos

GET /productos/{producto}/precios-ajax
Retorna: HTML con tabla de precios por lista
Uso: Modal en index de productos

GET /productos/{producto}/stock-ajax
Retorna: HTML con información de stock actual
Uso: Modal en index de productos
```

5.2 Endpoints AJAX - Catálogo

```
POST /catalogo/productos
Body: {categoria_id?, search?, page}
Retorna: JSON con productos paginados
Uso: Carga de productos en catálogo con filtros

GET /catalogo/producto/{producto}
Retorna: HTML con detalle completo del producto
Uso: Modal de producto en catálogo

POST /catalogo/solicitud
Body: {cliente_id, enlace_id?, items[], notas}
Retorna: JSON {success, numero_solicitud, mensaje}
Uso: Envío de cotización desde catálogo
```

5.3 Endpoints AJAX - Stock

```
GET /stock/productos-json?term={busqueda}
Retorna: JSON [{id, text, referencia, nombre, tiene_variantes}, ...]
Uso: Autocompletado en operaciones de stock

GET /stock/{id}/obtener
Retorna: JSON con stock actual del producto
Uso: Validación antes de operaciones

GET /stock/historial?producto_id={id}&fecha_desde={date}&fecha_hasta={date}
Retorna: HTML con tabla de movimientos
Uso: Ver historial filtrado
```

5.4 Endpoints AJAX - Geografía

```
GET /ajax/ciudades?departamento_id={id}
Retorna: JSON [{id, nombre}, ...]
Uso: Carga dinámica de ciudades al seleccionar departamento
```

5.5 Endpoints AJAX - Servicio Técnico

```
GET /servicio-tecnico/equipos-cliente/{clienteId}
Retorna: JSON [{id, descripcion}, ...]
Uso: Carga de equipos al seleccionar cliente en formulario de orden

POST /servicio-tecnico/ordenes/{orden}/cambiar-estado
Body: {estado, observaciones}
Retorna: JSON {success, mensaje}
Uso: Actualización de estado de orden

POST /servicio-tecnico/ordenes/{orden}/diagnosticos
Body: {datos del diagnóstico}
Retorna: JSON {success, mensaje}
Uso: Agregar diagnóstico a orden

POST /servicio-tecnico/ordenes/{orden}/repuestos
Body: {repuesto_id, cantidad}
Retorna: JSON {success, mensaje}
Uso: Agregar repuesto a orden (descuenta stock)
```

5.6 Endpoints de Exportación

```
GET /solicitudes/{solicitud}/pdf
Retorna: PDF (inline o download)
Uso: Generar PDF de cotización

GET /solicitudes/exportar-excel?fecha_desde={date}&fecha_hasta={date}
Retorna: Archivo Excel
Uso: Exportar listado de cotizaciones

GET /servicio-tecnico/ordenes/{orden}/pdf
Retorna: PDF de orden de servicio
Uso: Generar PDF de orden

GET /stock/exportar-movimientos?fecha_desde={date}&fecha_hasta={date}
Retorna: Archivo Excel
Uso: Exportar historial de movimientos
```

5.7 Endpoints de Importación

```
POST /productos/actualizar-precios-excel
Body: FormData {archivo (CSV/Excel)}
Retorna: JSON {success, mensaje, estadisticas}
Formato CSV: Delimitador ';'
Columnas: referencia;export1;export2;local1;local2;local3;local4
Uso: Actualización masiva de precios

POST /productos/importar
Body: FormData {archivo (CSV/Excel)}
Retorna: JSON {success, mensaje, estadisticas}
Uso: Importación masiva de productos
```

5.8 DataTables Server-Side

Todos los índices usan DataTables con procesamiento server-side:

```
GET /productos?draw={n}&start={n}&length={n}&search[value]={text}&order[0][column]={n}
GET /clientes?...
GET /solicitudes?...
GET /stock?...
GET /servicio-tecnico/ordenes?...
GET /servicio-tecnico/clientes?...
GET /servicio-tecnico/equipos?...
GET /servicio-tecnico/repuestos?...

Retorna: JSON DataTables format
{
  draw: int,
  recordsTotal: int,
  recordsFiltered: int,
  data: [...]
}
```

6. SEGURIDAD Y AUTENTICACIÓN

6.1 Autenticación

Sistema: Laravel Breeze (sesiones)

Rutas de autenticación:

```
GET|POST /login      - Inicio de sesión
POST /logout         - Cierre de sesión
GET|POST /register    - Registro (deshabilitado en producción)
GET|POST /forgot-password - Recuperación de contraseña
GET|POST /reset-password - Reseteo de contraseña
```

Características:

- Hash de contraseñas con bcrypt
- Protección CSRF en todos los formularios
- Throttling de intentos de login
- Remember me functionality
- Email verification (opcional)

6.2 Autorización

Sistema: Spatie Laravel Permission

Roles del Sistema:

1. **admin:** Acceso total
2. **vendedor:** Acceso a módulos de catálogo y ventas
3. **tecnico:** Acceso a módulo de servicio técnico

Matriz de Permisos:

Módulo	Admin	Vendedor	Técnico
Dashboard	OK	OK	OK
Métricas	OK	X	X
Usuarios	OK	X	X
Categorías	OK	X	X
Productos	OK	X	X
Clientes (B2B)	OK	OK*	X
Enlaces	OK	OK*	X
Catálogo	OK	OK	X
Cotizaciones	OK	OK*	X
Stock	OK	OK	X
Servicio Técnico	OK	X	OK

*Vendedores solo ven sus clientes asignados

Implementación en Rutas:

```
Route::middleware(['auth'])->group(function () {  
    // Rutas protegidas  
});  
  
// Verificación en controladores  
if (auth()->user()->hasRole('admin')) {  
    // Acción de admin  
}  
  
// Filtrado por vendedor  
if (auth()->user()->hasRole('vendedor')) {  
    $clientes = Cliente::where('vendedor_id', auth()->id()->get());  
}
```

6.3 Acceso de Clientes (Token)

Sistema: Enlaces temporales con tokens únicos

Características:

- Token aleatorio de 32 caracteres
- Fecha de expiración configurable
- Control de activación manual
- Registro de visitas y último acceso
- Sin necesidad de crear cuenta de usuario

Validación:

```
public function esValido(): bool
{
    return $this->activo
        && $this->expira_en->isFuture();
}
```

6.4 Protección CSRF

Todos los formularios incluyen token CSRF:

```
<form method="POST" action="...">
    @csrf
    <!-- campos -->
</form>
```

6.5 Validación de Inputs

Server-side validation en todos los formularios:

```
$request->validate([
    'email' => 'required|email|unique:users',
    'password' => 'required|min:6',
    // ...
]);
```

Client-side validation con HTML5:

```
<input type="email" required>
<input type="number" min="0" max="100">
```

6.6 Protección XSS

- Blade escapa automáticamente: `{{ $variable }}`
- Para HTML confiable: `{!! $html !!}` (usar con precaución)
- Sanitización en inputs con `strip_tags()` donde sea necesario

6.7 Protección SQL Injection

- Uso exclusivo de Eloquent ORM y Query Builder
- Prepared statements automáticos
- No se ejecuta SQL raw sin bindings

6.8 Auditoría

Tabla `logs` registra acciones críticas:

- Creación/edición/eliminación de registros
- Cambios de estado
- Accesos a módulos sensibles
- IP del usuario

Tabla **movimientos_stock** auditoría completa:

- Todos los movimientos de stock
- Usuario responsable
- Stock antes/después
- Documento de referencia

Tabla **actualizaciones_precios** auditoría de precios:

- Archivo subido
- Resultados del procesamiento
- Errores detallados
- Usuario responsable

7. WORKFLOWS TÉCNICOS

7.1 Workflow: Creación de Producto

```
1. Admin accede a /productos/form
2. Completa información básica
  |--? referencia (único)
  |--? nombre, descripción, marca
  |--? categoria_id (select)
  |--? unidades (venta, empaque)
3. Decide si tiene variantes
  |--? NO: Configura stock inicial
  |--? Sí: Agrega filas de variantes (talla, color, sku)
4. Sube imágenes (marca una como principal)
5. Ingresar precios por lista (opcional)
6. Submit -> ProductosController@guardar
7. Backend validation
8. Guarda producto en DB
9. Si controlar_stock = true:
  |--? Crea registro(s) en stock_productos
  |--? Si stock_inicial > 0: crea MovimientoStock (tipo: ajuste, origen:
ajuste_inventario)
10. Guarda variantes (si aplica)
11. Procesa imágenes:
  |--? Mueve a storage/app/public/imagenes/productos/{producto_id}/
  |--? Guarda registros en imagenes_producto
12. Guarda precios (si aplica)
13. Redirect a /productos con mensaje éxito
```


7.2 Workflow: Actualización Masiva de Precios

```
1. Admin descarga plantilla CSV/Excel
   |__? /productos/descargar-plantilla-csv o -excel
2. Llena archivo con referencias y nuevos precios
   Formato: referencia;export1;export2;local1;local2;local3;local4
3. Sube archivo en /productos (sección actualización precios)
4. ProductosController@actualizarPreciosExcel
5. Valida archivo (formato, extensión, tamaño)
6. Crea registro en actualizaciones_precios (estado: procesando)
7. Guarda archivo en storage/app/public/uploads/actualizaciones_precios/
8. Ejecuta PreciosImport (Maatwebsite Excel)
9. Por cada fila:
   |--? Busca producto por referencia
   |--? Si no existe: registra error
   |__? Si existe:
       |--? Por cada lista de precio con valor:
       |   |--? Busca o crea registro en precios_productos
       |   |__? Actualiza precio
       |__? Registra éxito
10. Actualiza registro actualizaciones_precios:
    |--? estado = completado/error
    |--? total_filas, exitosas, fallidas
    |--? errores (JSON)
    |__? detalles_procesados (JSON)
11. Retorna JSON con estadísticas
12. Frontend muestra resumen con SweetAlert2
```

7.3 Workflow: Cliente Solicita Cotización (Flujo A)

```
1. Vendedor crea Cliente en /clientes
2. Vendedor crea Enlace en /enlaces
   |--? Selecciona cliente
   |--? Configura días_validos, mostrar_precios, mostrar_stock
   |__? Sistema genera token único
3. Sistema envía email a cliente con URL:
   /catalogo/{token}
4. Cliente hace clic en enlace
5. CatalogoController@show valida:
   |--? Token existe
   |--? Enlace activo
   |--? No expirado
   |__? Si válido: incrementa visitas, actualiza ultimo_acceso
6. Cliente navega catálogo:
   |--? Ve productos según categoría
   |--? Ve precios si mostrar_precios = true
   |__? Ve stock si mostrar_stock = true
7. Cliente agrega productos a carrito (session)
8. Cliente envía cotización
9. POST /catalogo/solicitud
10. Backend crea SolicitudCotizacion:
    |--? Genera numero_solicitud (SC-YYYYMMDDHHMMSS-XXXX)
    |--? estado = pendiente
    |--? cliente_id, enlace_acceso_id
    |__? monto_total calculado
11. Backend crea ItemSolicitudCotizacion por cada producto
12. Sistema envía email a vendedor asignado
13. Retorna JSON con número de solicitud
14. Frontend muestra confirmación
```

7.4 Workflow: Revisión y Aprobación de Cotización

```
1. Vendedor/Admin accede a /solicitudes
2. Ve listado de cotizaciones (filtradas por vendedor si aplica)
3. Hace clic en "Ver detalle"
4. SolicitudController@detalle muestra:
  |--? Información del cliente
  |--? Items cotizados (producto, cantidad, precio)
  |--? Monto total
  |--? Notas del cliente
  |--? Disponibilidad de stock (en tiempo real)
  |__? Acciones disponibles
5. Vendedor decide:

  OPCIÓN A: Aprobar
  |--? Hace clic en "Aplicar Cotización"
  |--? Modal pregunta si descontar stock
  |--? POST /solicitudes/{id}/aplicar
  |--? Backend:
  |   |--? Actualiza solicitud: estado = aplicada, aplicada_en = now(),
  aplicada_por = auth()->id()
  |   |__? Si descontar_stock = true:
  |       |--? Por cada item:
  |           |--? Valida stock disponible
  |           |--? Crea MovimientoStock (tipo: salida, origen: venta)
  |           |__? Actualiza stock_productos (descuenta cantidad)
  |       |__? Vincula movimientos con solicitud_cotizacion_id
  |--? Envía email al cliente (cotización aprobada)
  |__? Retorna JSON success

  OPCIÓN B: Rechazar
  |--? Hace clic en "Rechazar Cotización"
  |--? Modal solicita motivo
  |--? POST /solicitudes/{id}/rechazar
  |--? Backend:
  |   |--? Actualiza solicitud: estado = rechazada, rechazada_en = now(),
  |   |   rechazada_por = auth()->id(), motivo_rechazo = motivo
  |   |__? Envía email al cliente (cotización rechazada con motivo)
  |   |__? Retorna JSON success

6. Frontend actualiza vista con nuevo estado
```

7.5 Workflow: Orden de Servicio Técnico

```
1. Cliente trae equipo al taller
2. Técnico/Admin accede a /servicio-tecnico/ordenes/form
3. Busca cliente:
  |--? Si existe: selecciona
  |__? Si no existe: crea nuevo en modal (/servicio-tecnico/clientes/create)
4. Selecciona cliente -> AJAX carga equipos del cliente
5. Busca equipo:
  |--? Si está registrado: selecciona
  |__? Si no: puede crear nuevo en modal (/servicio-tecnico/equipos/create)
6. Completa información de orden:
  |--? tipo_servicio (Reparación, Mantenimiento, etc.)
  |--? prioridad (baja, media, alta, urgente)
  |--? descripcion_problema (detallado)
  |--? accesorios_entregados
  |--? fecha_recepcion (default: hoy)
  |--? fecha_promesa_entrega (opcional)
  |__? tecnico asignado (opcional)
7. Submit -> STOrdenServicioController@store
8. Backend:
  |--? Genera numero_orden (ST-YYYY-XXXXXX)
  |--? estado = 'recibida' (si no hay técnico) o 'asignada' (si hay técnico)
  |--? user_id = auth()->id()
  |--? Guarda orden
  |__? Si tecnico_id presente:
    |--? Crea registro en st_historial_estados (recibida -> asignada)
    |__? Envía email al técnico
9. Redirect a detalle de orden
10. Técnico accede a orden:
  |--? Sube imágenes de recepción
  |--? Inicia trabajo (cambia estado a 'en_proceso')
  |--? Agrega diagnóstico:
    | |--? Modal /servicio-tecnico/ordenes/{id}/diagnosticos
    | |--? Completa diagnóstico técnico
    | |__? Marca si requiere repuestos
  |--? Si requiere repuestos:
    | |--? Verifica stock en /servicio-tecnico/repuestos
    | |--? Si hay stock: agrega repuestos a orden
    | | |__? POST /servicio-tecnico/ordenes/{id}/repuestos
    | | | |--? Descuenta de st_repuestos.stock_actual
    | | | |--? Crea registro en st_repuestos_usados
    | | |__? Actualiza costo_repuestos y costo_total de orden
    | |__? Si no hay stock: cambia estado a 'pendiente_repuestos'
  |--? Realiza reparación
  |--? Sube imágenes de reparación
  |--? Actualiza costo_mano_obra
  |__? Marca como 'completada'
11. Notifica a cliente (email/SMS)
12. Cliente recoge equipo
13. Técnico marca como 'entregada'
14. Sistema registra fecha_entrega
```

7.6 Workflow: Movimiento de Stock

```
ENTRADA DE STOCK:
1. Usuario accede a /stock
2. Clic en "Registrar Entrada"
3. Modal muestra formulario
4. Busca producto (autocomplete AJAX)
5. Si tiene variantes: selecciona variante
6. Ingresa:
  |--? cantidad (positiva)
  |--? tipo: compra o devolucion
  |--? referencia_documento (ej: Factura #12345)
  |--? motivo (opcional)
  |--? ubicacion (opcional)
7. Submit -> StockController@entrada
8. Backend:
  |--? Busca registro en stock_productos
  |--? stock_anterior = cantidad_disponible actual
  |--? stock_nuevo = stock_anterior + cantidad
  |--? Actualiza stock_productos.cantidad_disponible
  |--? Crea MovimientoStock:
  |   |--? tipo_movimiento = 'entrada'
  |   |--? origen = 'compra' o 'devolucion'
  |   |--? cantidad, stock_anterior, stock_nuevo
  |   |--? usuario_id = auth()->id()
  |   |--? referencia_documento, motivo
  |   |--? Retorna JSON success
9. Frontend actualiza tabla y cierra modal

SALIDA DE STOCK:
Similar a entrada, pero:
- Valida que stock_disponible >= cantidad
- stock_nuevo = stock_anterior - cantidad
- tipo_movimiento = 'salida'
- origen = 'venta' o 'transferencia'

AJUSTE DE STOCK:
- Permite stock positivo o negativo
- tipo_movimiento = 'ajuste'
- origen = 'ajuste_inventario'
- Usado para correcciones de inventario físico
```

8. COMANDOS DE DESARROLLO

8.1 Instalación Inicial

```
# Clonar repositorio
git clone <repository-url>
cd portfolio

# Instalar dependencias PHP
composer install

# Instalar dependencias Node
npm install

# Copiar archivo de configuración
copy .env.example .env

# Generar app key
php artisan key:generate

# Configurar base de datos en .env
# DB_CONNECTION=mysql
# DB_HOST=127.0.0.1
# DB_PORT=3306
# DB_DATABASE=nombre_bd
# DB_USERNAME=root
# DB_PASSWORD=

# Iniciar XAMPP (MySQL + Apache)

# Ejecutar migraciones
php artisan migrate

# Ejecutar seeders (opcional)
php artisan db:seed

# Crear symlink para storage
php artisan storage:link

# Compilar assets
npm run build
```

8.2 Desarrollo Diario

```
# Terminal 1: Iniciar XAMPP
# (Interfaz gráfica de XAMPP)

# Terminal 2: Laravel dev server
php artisan serve
# Acceso: http://localhost:8000

# Terminal 3: Vite dev server (hot reload)
npm run dev
# Vite corre en http://localhost:5173
```

8.3 Base de Datos

```
# Ejecutar migraciones pendientes
php artisan migrate

# Rollback última migración
php artisan migrate:rollback

# Rollback todas las migraciones
php artisan migrate:reset

# Rollback + migrate (refresh)
php artisan migrate:refresh

# Fresh: Drop todas las tablas + migrate
php artisan migrate:fresh

# Fresh + seeders
php artisan migrate:fresh --seed

# Ejecutar seeder específico
php artisan db:seed --class=RolesSeeder

# Crear nueva migración
php artisan make:migration create_tabla_nombre

# Crear modelo + migración
php artisan make:model NombreModelo -m

# Crear modelo + migración + controller + factory + seeder
php artisan make:model NombreModelo -a
```

8.4 Testing

```
# Ejecutar todos los tests
php artisan test

# Con coverage
php artisan test --coverage

# Test específico
php artisan test --filter NombreTest

# PHPUnit directo
vendor/bin/phpunit

# Con configuración específica
vendor/bin/phpunit --configuration phpunit.xml
```

8.5 Cache y Optimización

```
# Limpiar todos los caches
php artisan optimize:clear

# Limpiar cache de configuración
php artisan config:clear

# Limpiar cache de rutas
php artisan route:clear

# Limpiar cache de vistas
php artisan view:clear

# Cachear configuración (producción)
php artisan config:cache

# Cachear rutas (producción)
php artisan route:cache

# Cachear vistas (producción)
php artisan view:cache
```

8.6 Assets y Build

```
# Desarrollo (watch mode)
npm run dev

# Build producción
npm run build

# Limpiar node_modules y reinstalar
rm -rf node_modules
npm install
```

8.7 Comandos Personalizados del Proyecto

```
# Inicializar stock de productos sin stock
# (Crea registros en stock_productos para productos con controlar_stock=true)
php artisan stock:inicializar

# Limpiar enlaces expirados
php artisan enlaces:limpiar-expirados

# Generar reporte de stock bajo (email)
php artisan stock:reporte-bajo
```

8.8 Mantenimiento

```
# Ver rutas registradas
php artisan route:list

# Buscar ruta específica
php artisan route:list --path=productos

# Ver rutas de un método
php artisan route:list --method=GET

# Información de la aplicación
php artisan about

# Modo mantenimiento ON
php artisan down

# Modo mantenimiento OFF
php artisan up

# Ver logs en tiempo real (requiere tail)
tail -f storage/logs/laravel.log
```


FIN DE LA DOCUMENTACION TECNICA

Sistema: Portfolio B2B + Servicio Tecnico

Framework: Laravel 9.52

Version: 1.0